CHƯƠNG 6
TẦNG VẬN CHUYỂN

Bản tóm tắt để đọc nhanh và ôn thi
Nguồn chính: slide-vi 06 - Transport Layer
Bổ sung: slide-en Chapter 3 (checksum, RTT, congestion, QUIC)

1 Chức năng, dồn/phân kênh	2 Truyền tin cậy (RDT)	3 Pipeline: GBN, SR
4 TCP	5 Tắc nghẽn	6 UDP
★ Ôn tập và câu hỏi		

Mục tiêu bài học

- Giải thích chức năng của tầng transport: kênh logic giữa 2 process, dồn kênh và phân kênh bằng port.
- Mô tả các phiên bản rdt 1.0 → 3.0 và hai giải pháp pipeline Go-Back-N, Selective Repeat.
- Nắm cấu trúc segment TCP, bắt tay 3 bước, điều khiển luồng, tắc nghẽn; so sánh với UDP.

Chú thích màu: khối tím là nội dung bổ sung từ slide tiếng Anh. Chỗ ghi **[cần kiểm tra lại]** là điểm slide-vi chỉ có hình/FSM không đọc được rõ.

1 Chức năng và dồn kênh - phân kênh

- Tầng transport cung cấp **kênh truyền logic giữa 2 tiến trình** trên 2 máy; chỉ chạy ở **end-system** (router chỉ xử lý đến tầng network).
- **Bên gửi - dồn kênh (multiplexing):** nhận dữ liệu từ các socket, chia thông điệp thành segment, dán header (source port, dest port), chuyển xuống tầng network.
- **Bên nhận - phân kênh (demultiplexing):** nhận segment từ tầng network, bóc header và chuyển dữ liệu lên đúng socket dựa vào port.
- Tầng transport hỗ trợ: truyền tin cậy (điều khiển luồng, điều khiển tắc nghẽn, thiết lập và duy trì kết nối) hoặc truyền không tin cậy (nỗ lực tốt nhất); **không** đảm bảo độ trễ và băng thông.

2 Nguyên tắc truyền dữ liệu đáng tin cậy (RDT)

- **RDT** = Reliable Data Transfer; nguyên tắc **dừng và chờ** (stop-and-wait): gửi một gói kèm thông tin kiểm lỗi (checksum, parity, CRC), chờ ACK rồi mới gửi tiếp; gửi lại khi lỗi bit hoặc mất gói.
- Tín hiệu phản hồi: **ACK** (nhận đúng), **NAK** (nhận sai). Cơ chế dùng: checksum, ACK, NAK, số thứ tự (1 bit: 0 hoặc 1), timer.

Phiên bản	Giả thiết kênh	Cơ chế thêm vào
rdt 1.0	Kênh lý tưởng: không lỗi bit, không mất gói	Gửi và nhận trực tiếp (FSM đơn giản)
rdt 2.0	Có lỗi bit	Checksum, ACK/NAK; gửi lại khi nhận NAK. Lỗ hổng: ACK/NAK cũng có thể sai
rdt 2.1	Lỗi bit, ACK/NAK có thể sai	Số thứ tự 0/1 cho gói; bên gửi gửi lại khi ACK/NAK lỗi; bên nhận loại gói trùng bằng cách so số thứ tự với trạng thái đang chờ
rdt 2.2	Như 2.1, bỏ NAK	Bên nhận ACK gói đúng cuối cùng kèm số thứ tự; ACK trùng được coi là NAK
rdt 3.0	Lỗi bit và mất gói	Thêm **timer**: không có ACK trong thời gian hợp lý thì gửi lại; gói/ACK trễ gây gói trùng, đã xử lý bằng số thứ tự

Hiệu quả của rdt 3.0 (ví dụ slide): đường truyền 1 Gbps, trễ end-to-end 15 ms, gói 8 Kb. Thời gian truyền một gói d_{trans} = L/R = 8000 bit / 10^9 bit/s = 8 µs; RTT = 2 × 15 ms = 30 ms.

$U_{sender} = (L/R) / (RTT + L/R)$	$U_{sender} = 0{,}008 / 30{,}008 \approx 0{,}00027$
Tỉ lệ thời gian bên gửi thực sự gửi	≈ 0,027%: dừng và chờ lãng phí đường truyền

Ký hiệu: L kích thước gói (bit), R tốc độ đường truyền (bit/s). Slide chỉ ghi "nghi thức đã hạn chế việc sử dụng tài nguyên mạng"; phép tính trên do mình tính lại từ số liệu slide.

3 Pipeline: Go-Back-N và Selective Repeat

- **Pipeline:** cho gửi nhiều gói khi chưa nhận ACK; số thứ tự tăng dần; dùng buffer (sliding window) ở bên gửi và/hoặc bên nhận. Với 3 gói liên tiếp, hiệu quả sử dụng tăng 3 lần (theo slide).

	Go-Back-N (GBN)	Selective Repeat (SR, gửi lại có chọn)
Cửa sổ	Cửa sổ gửi N (số thứ tự k-bit); số gói gửi liên tục chưa ACK	Cả bên gửi và bên nhận có cửa sổ
ACK	**ACK tích lũy**: ACK(n) là nhận đúng đến n; chỉ ACK gói đúng thứ tự cao nhất	**ACK riêng lẻ** từng gói nhận đúng
Bên nhận	Gói sai thứ tự: **loại bỏ** (không buffer), gửi lại ACK lớn nhất	Gói sai thứ tự: **lưu buffer** nếu còn chỗ; chuyển lên theo thứ tự
Timer	1 đồng hồ cho gói cũ nhất (đầu cửa sổ)	Mỗi gói chưa ACK có đồng hồ riêng
Khi timeout	Gửi lại **tất cả** gói chưa ACK trong cửa sổ	Chỉ gửi lại gói không nhận được ACK

Slide nêu câu hỏi: với số thứ tự 0,1,2,3 và window size 3 thì quan hệ giữa kích thước số thứ tự và window size là gì? Slide không ghi đáp án; điều kiện thường dùng cho SR là kích thước window ≤ nửa không gian số thứ tự **[cần kiểm tra lại]**.

4 Giao thức TCP

- **TCP** [RFC 793, 1122, 1323, 2018, 2581]: **point-to-point** (1 gửi, 1 nhận), **full-duplex**, hướng kết nối (handshake trước khi gửi), dòng byte (stream-of-bytes) không có ranh giới gói, tin cậy và theo thứ tự, pipeline, có kiểm soát luồng và tắc nghẽn; dùng buffer gửi và nhận; **MSS** là kích thước segment tối đa.
- **Cấu trúc segment:** source port, dest port, **sequence number** (số thứ tự của byte đầu tiên trong data), **acknowledgment number** (số thứ tự byte đang mong nhận tiếp), độ dài header, các cờ **U A P R S F** (URG, ACK, PSH, RST, SYN, FIN), **receive window** (số byte bên nhận sẵn sàng nhận), checksum, urgent pointer, options, dữ liệu.
- **Cờ:** SYN thiết lập kết nối; FIN đóng kết nối; ACK báo trường ack hợp lệ; RST reset kết nối; PSH phân phối ngay; URG dữ liệu khẩn (ít dùng).

Truyền tin cậy trong TCP

- **Bên gửi:** nhận dữ liệu từ ứng dụng, tạo segment, bật đồng hồ (cho gói đầu buffer). Nhận ACK mới thì trượt cửa sổ và đặt lại đồng hồ. **Timeout**: gửi lại dữ liệu còn trong buffer rồi reset đồng hồ.
- **Bên nhận:** nhận đúng thứ tự thì chấp nhận và ACK; nhận sai thứ tự (phát hiện "khoảng trống" GAP) thì gửi **ACK trùng**.
- Ví dụ telnet trong slide: người dùng gõ ký tự 'C', host kia ACK và echo lại 'C', rồi host đầu ACK ký tự echo; Seq là số thứ tự byte đầu của vùng data, ACK là số byte chờ nhận tiếp (giá trị cụ thể trong hình **[cần kiểm tra lại]**).

BỔ SUNG TỪ SLIDE-EN · RTT, timeout, ACK và fast retransmit (slide-en 81-88)

- Timeout phải **dài hơn RTT** nhưng RTT thay đổi: quá ngắn gây timeout sớm, gửi lại thừa; quá dài thì phản ứng chậm khi mất gói. **SampleRTT** là thời gian đo từ lúc gửi segment đến khi nhận ACK (bỏ qua segment gửi lại).
- **EstimatedRTT = (1 − α)·EstimatedRTT + α·SampleRTT** (EWMA), α điển hình = **0,125**. **TimeoutInterval = EstimatedRTT + 4·DevRTT**, với **DevRTT** là EWMA của độ lệch |SampleRTT − EstimatedRTT| (hệ số β = 0,25 là giá trị chuẩn, slide chỉ hiện trong ảnh công thức **[cần kiểm tra lại]**).
- Quy tắc ACK của bên nhận (RFC 5681): gói đúng thứ tự → **delayed ACK** chờ tối đa 500 ms; đang có 1 gói chờ ACK mà đến thêm gói đúng thứ tự → gửi ngay **ACK tích lũy** cho cả hai; gói sai thứ tự (seq cao hơn mong đợi, có gap) → gửi ngay **ACK trùng**; gói lấp (một phần hoặc toàn bộ) khoảng trống ở đầu gap → ACK ngay.
- **Fast retransmit**: nhận **3 ACK trùng** cho cùng dữ liệu thì gửi lại segment bị coi là mất mà không đợi timeout (nội dung slide 88 là hình, theo kiến thức chương này **[cần kiểm tra lại]**).

Quản lý kết nối

- **Thiết lập: bắt tay 3 bước** (three-way handshake): client gửi SYN → server trả SYN+ACK (trạng thái SYN_RCVD) → client gửi ACK; hai bên vào trạng thái ESTABLISHED.
- **Đóng kết nối:** slide-vi ghi "bắt tay 2 lần" mỗi chiều: bên gửi FIN, bên kia ACK; client đi qua **timed wait** trước khi closed. Các sơ đồ vòng đời client/server là hình ảnh **[cần kiểm tra lại]** chi tiết trạng thái.

Điều khiển luồng

- Tránh bên gửi làm tràn bộ đệm bên nhận. Bên nhận báo trường **window size** (lượng dữ liệu còn nhận được vào buffer). Ví dụ slide: buffer bên nhận 4K; ứng dụng gửi 2K → còn trống 2K nên bên gửi gửi được tối đa 2K; khi buffer đầy (window = 0) thì bên gửi bị khóa; khi ứng dụng đọc dữ liệu thì window mở lại.

5 Kiểm soát tắc nghẽn

- **Tắc nghẽn:** một node nhận dữ liệu từ nhiều nguồn, buffer giới hạn, gói đến ồ ạt không xử lý kịp → mất gói, delay cao, dùng đường truyền kém hiệu quả. Khác với điều khiển luồng (bảo vệ bên nhận).
- TCP: bên gửi đặt tốc độ gửi dựa trên phản hồi (nhận ACK, mất gói, độ trễ). Có **2 pha: Slow-Start** và **Congestion Avoidance**.

BỔ SUNG TỪ SLIDE-EN · Congestion control chi tiết (slide-en 105-134)

- **Nguyên nhân và chi phí:** trễ lớn khi tốc độ đến tiến gần dung lượng R/2 (kịch bản 1); gửi lại do mất gói và gửi lại thừa do timeout sớm làm giảm throughput hữu ích (kịch bản 2); gói bị rớt ở hop sau làm lãng phí tài nguyên các hop trước đó (kịch bản 3).
- Hai hướng tiếp cận: **end-end** (không có phản hồi tường minh từ mạng, suy từ mất gói và delay - cách TCP dùng) và **network-assisted** (router báo, ví dụ ECN).
- **AIMD** (additive increase, multiplicative decrease): tăng cwnd 1 MSS mỗi RTT cho đến khi có mất gói rồi giảm; mất gói do **3 ACK trùng** → cắt cwnd một nửa (TCP Reno); do **timeout** → về 1 MSS (TCP Tahoe, theo slide).
- **Slow start:** cwnd khởi đầu = 1 MSS, tăng gấp đôi mỗi RTT (cộng 1 MSS cho mỗi ACK). Chuyển sang congestion avoidance (tăng tuyến tính) khi cwnd đạt **ssthresh**; khi mất gói, ssthresh = cwnd/2 tại thời điểm trước mất gói.
- **TCP CUBIC** (mặc định ở Linux): đi nhanh tới W_{max} (tốc độ lúc mất gói) rồi tiến chậm lại gần W_{max}. **Delay-based** (ví dụ BBR của Google): giữ "ống vừa đầy" bằng cwnd/RTT_{min}. **ECN**: router đánh dấu 2 bit trong IP header, bên nhận báo bằng cờ ECE.
- **Công bằng:** K phiên TCP cùng bottleneck R thì mỗi phiên R/K. UDP đa phương tiện không bị kiểm soát tắc nghẽn; mở nhiều kết nối TCP song song chiếm nhiều băng thông hơn (ví dụ link có 9 kết nối: mở thêm 1 kết nối được R/10, mở 11 kết nối được R/2).
- **Throughput TCP trung bình ≈ 3/4 · W / RTT** (W là cửa sổ tại lúc mất gói). Ví dụ "long fat pipe": segment 1500 byte, RTT 100 ms, muốn 10 Gbps → cần W = 83.333 segment in-flight, tương ứng loss rate $L = 2 \cdot 10^{-10}$ (công thức Mathis theo slide).

6 Giao thức UDP

- **UDP** (User Datagram Protocol) [RFC 768]: dịch vụ "nỗ lực tốt nhất", gói có thể **mất** hoặc **sai thứ tự**; **không kết nối**, không handshake, mỗi gói xử lý độc lập, không có trạng thái kết nối.
- **Header UDP** chỉ 4 trường 32 bit: source port, dest port, length (tính cả header), checksum, rồi dữ liệu.
- **Vì sao dùng UDP:** không cần thiết lập kết nối, đơn giản (không quản lý trạng thái), không kiểm soát luồng, header nhỏ, nhanh. Muốn tin cậy trên UDP thì **tầng ứng dụng** tự phát hiện và phục hồi lỗi.
- **Ứng dụng thường dùng:** multimedia (chịu lỗi, cần tốc độ), DNS, SNMP, TFTP.

BỔ SUNG TỪ SLIDE-EN · UDP checksum và QUIC (slide-en 31-34, 136-140)

- **Checksum:** bên gửi coi segment (gồm header UDP và địa chỉ IP) là dãy số nguyên 16 bit, cộng theo **one's complement** (có nhớ tràn bit cao nhất cộng vòng lại, wraparound), rồi lấy bù. Bên nhận tính lại; khác → có lỗi; bằng → chưa chắc không lỗi (bảo vệ yếu).
- Ví dụ: 1110011001100110 + 1101010101010101 = 1 1011101110111011 → cộng nhớ vòng: 1011101110111100 → lấy bù (checksum): **0100010001000011** (khớp với slide).
- **QUIC:** giao thức tầng ứng dụng chạy trên UDP (Google, dùng cho HTTP/3); thiết lập kết nối, mã hóa và xác thực chỉ trong **1 RTT** (thay vì 2 handshake nối tiếp TCP + TLS); nhiều stream cùng một kết nối, không bị HOL blocking giữa các stream.

★ Ôn tập nhanh

Cần thuộc

TCP handshake SYN → SYN+ACK → ACK	**EstimatedRTT** (1−α)·Est + α·Sample; α = 0,125 (slide-en)	**Slow start** cwnd 1 MSS, gấp đôi mỗi RTT tới ssthresh	**UDP header** src port, dst port, length, checksum

Lỗi thường gặp

Dễ nhầm	Cần nhớ
Điều khiển luồng và tắc nghẽn	Luồng: bảo vệ bên nhận (window size). Tắc nghẽn: bảo vệ mạng (cwnd)
GBN và SR	GBN: ACK tích lũy, timeout gửi lại cả cửa sổ, bên nhận không buffer. SR: ACK riêng, chỉ gửi lại gói mất, có buffer
Sequence number và ACK number của TCP	Seq = số thứ tự byte đầu của data; ACK = số của byte đang mong nhận tiếp
TCP có "ranh giới gói"	Không: là dòng byte; UDP mới giữ ranh giới từng datagram
Timeout vs 3 ACK trùng	Timeout: về 1 MSS (Tahoe); 3 ACK trùng: giảm một nửa (Reno) (slide-en)
UDP "không tin cậy"	Tầng ứng dụng có thể tự tạo độ tin cậy trên UDP (ví dụ QUIC)

Câu hỏi ôn tập (che cột đáp án rồi tự trả lời)

Câu hỏi	Đáp án
1. Dồn kênh và phân kênh khác nhau thế nào?	Dồn kênh ở bên gửi (gom dữ liệu từ các socket, gắn header port); phân kênh ở bên nhận (đưa segment đến đúng socket)
2. rdt 3.0 thêm gì so với rdt 2.2?	Timer để xử lý mất gói
3. Vì sao stop-and-wait kém hiệu quả?	Bên gửi chờ ACK cả RTT: ví dụ U ≈ 0,027% với 1 Gbps, RTT 30 ms, gói 8 Kb
4. Nêu khác biệt chính giữa GBN và SR.	GBN ACK tích lũy và gửi lại cả cửa sổ; SR ACK riêng và chỉ gửi lại gói mất
5. Ý nghĩa 6 cờ của TCP?	URG khẩn, ACK ack hợp lệ, PSH đẩy ngay, RST reset, SYN thiết lập, FIN đóng
6. Nêu 3 bước bắt tay TCP.	SYN → SYN+ACK → ACK
7. Bên nhận TCP làm gì khi thấy gap?	Gửi ACK trùng (số thứ tự byte đang mong nhận)
8. Hai pha của kiểm soát tắc nghẽn TCP?	Slow start và congestion avoidance
9. Tại sao nhiều ứng dụng đa phương tiện dùng UDP?	Chịu lỗi, cần tốc độ, không muốn bị bóp tốc độ, header nhỏ, không cần kết nối
10. Tính checksum 1110011001100110 + 1101010101010101.	0100010001000011 (cộng, nhớ vòng, lấy bù) (slide-en)

Thực hành gợi ý

- Dùng Wireshark bắt kết nối TCP, chỉ ra 3 gói SYN, SYN+ACK, ACK và các giá trị seq/ack.
- Tự vẽ biểu đồ cwnd theo RTT: slow start đến ssthresh rồi tuyến tính, khi mất gói thì ssthresh = cwnd/2.